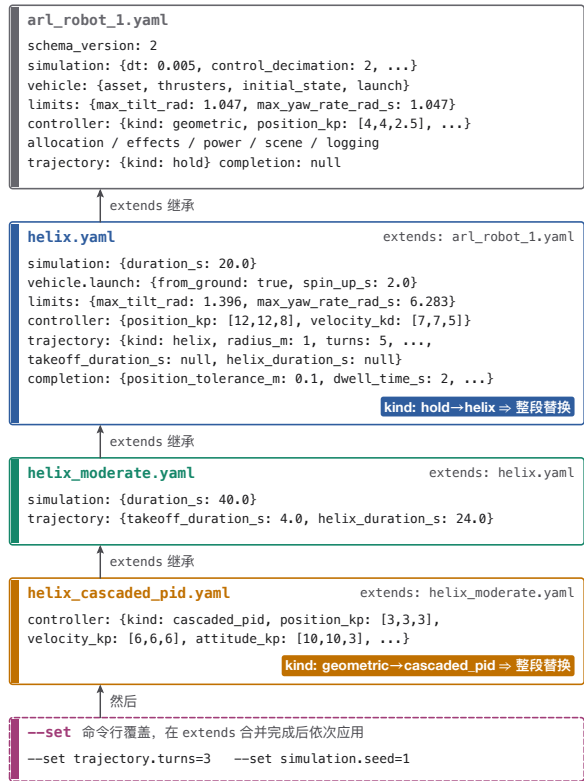


a 一次实验 = 一个继承父配置的 YAML 文件 + 命令行覆盖 configs/, config/loader.py, config/schema.py



合并规则 config/loader.py

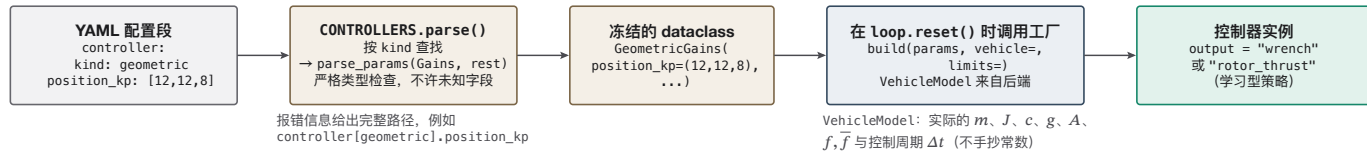
映射 + 映射 按键逐层递归合并
列表或标量 子文件的值整体替换父文件的值
kind: 不同 整段替换, 不会残留旧类型的增益
--set a,b=v v 按 YAML 解析; 父路径必须已存在
extends 环 直接报错 (Circular extends)

取值来源: arl_robot_1 helix helix_moderate helix_cascaded_pid --set



所有模块拿到的都是这份展开后的字典, 它写入 runs/<id>/config.json; metadata.json 另外记录 git commit、是否有未提交修改和 argv。
python -m isaac_drone validate 不启动 Isaac Sim 即可完成校验。

b 插件 = 参数 dataclass + 注册的工厂函数 core/registry.py, core/params.py



CONTROLLERS Registry

kind	参数 dataclass	说明
geometric	GeometricGains	Lee SO(3) 控制 + jerk/snap 前馈
cascaded_pid	CascadedPidGains	位置 → 速度 → 姿态 → 角速度

TRAJECTORIES Registry

kind	参数 dataclass	说明
hold	HoldParams	固定的质心目标与偏航
helix	HelixParams	起飞 + 螺旋, 可选最短时间规划

扩展点 (Python Protocol / 基类; 实现后注册或注入)

接口	定义位置	方法 / 约定	接入方式
Controller	control/base.py	reset · compute(state, setpoint, dt) · notify_allocation · diagnostics	注册表, 或 MotionControlLoop(controller=)
SegmentedTrajectory	trajectories/base.py	_build(p_0, ψ_0) → [Segment(setpoint(σ), length, $\Delta\psi$, T None)]	注册表; 自动获得最短时间规划、筛查、分阶段
EffectModel	effects/pipeline.py	reset(seed) · evaluate(state, dt, wind) → 绕质心的 Wrench	EffectsPipeline → MotionControlLoop(effects=)
StateProvider	sim/base.py	reset · read_state(t) → VehicleState (传感器 / 估计器)	MotionControlLoop(state_provider=)
PowerSystem	power/system.py	LoadModel.current_a · ActuatorEnvelope.thrust_bounds_n (需标定)	MotionControlLoop(power=)
RunHook	runtime/runner.py	start · after_step · paused · summary · close · finalize	run_experiment(hooks=)
VehicleBackend	sim/base.py	reset · read_state · motor_parameters · apply_motor_speeds · telemetry · ...	新后端 + SimulationDriver